FTE4312 HW1 文杰 123090612

Part A — True / False

1. SPV nodes validate PoW on headers and can request Merkle path proofs. T

Explanation: Simplified Payment Verification (SPV) nodes download only block headers to validate the Proof-of-Work (PoW) in the chain and request Merkle path proofs to verify specific transactions without running a full node.

2. In Bitcoin, miners prefer transactions with higher fee-per-byte when block space is limited. T

Explanation: Since blocks have a size limit, miners are motivated by an auction system to include transactions with the highest fees first to maximize their total rewards.

3. Bitcoin retargets difficulty every 2,016 blocks (a “period”). T

Explanation: All nodes in the network perform a decentralized retargeting of the PoW difficulty every 2,016 blocks to maintain a consistent 10-minute block generation rate.

4. In Ethereum, EOAs have contract code stored by default. F

Explanation: Externally Owned Accounts (EOAs) are controlled by private keys and do not have associated code; only Contract Accounts (CAs) contain and are governed by EVM code.

5. In Ethereum, storage is persistent, while memory is temporary for execution. T

Explanation: Storage acts like a hard drive for long-term data (account states), whereas memory is short-term, linear storage (like RAM) used only during the execution of a contract.

6. If a call has leftover gas, the unused gas is refunded (i.e., not charged). T

Explanation: If a transaction call completes with gas remaining, that leftover amount is refunded to the sender; however, if it runs out of gas, all changes are reverted and no gas is refunded.

7. Under EIP-1559, the base fee is burned and the priority fee is a tip. T

Explanation: Following the London Upgrade, the base fee is removed from circulation (burned), while the priority fee is given to miners as a "tip" to encourage them to include the transaction.

8. A transaction with an old nonce can still be executed again if it is re-broadcast. F

Explanation: The nonce is a counter that increments with each transaction; the system checks to ensure a nonce has not been used before to prevent "Replay Attacks".

9. Locktime can delay when a Bitcoin transaction becomes valid for mining/relay. T

Explanation: The Locktime field in a Bitcoin transaction specifies the earliest time or block height at which a transaction becomes valid for inclusion in the blockchain.

10. A Merkle root changes if any included transaction changes. T

Explanation: Because the Merkle root is the final hash produced by recursively hashing transaction data, tampering with any single transaction at the leaf level will propagate changes up to the root.

Part B — Multiple Choice

1. **A. 0.005 BTC**
   Explanation: In Bitcoin, **fee = sum(inputs) − sum(outputs)**, so $0.900 - 0.895 = 0.005$ BTC.

2. **B. fee ÷ transaction size in bytes**
   Explanation: “Fee-per-byte” (often sat/vB) means the fee normalized by the **transaction’s byte size**, since fees scale with tx size.

3. **B. They are probabilistic search filters**
   Explanation: Bloom filters are **probabilistic**: they can say “definitely not” or “maybe yes” (false positives possible).

4. **B. Priority fee**
   Explanation: Under EIP-1559, the **base fee is burned**, while the **priority fee (tip)** is paid to miners/validators as the main direct incentive.

5. **B. Nonce**
   Explanation: Ethereum uses the sender's **nonce** (transaction sequence number) to stop replays of the same signed transaction.

6. **B. Effects discarded; gas not refunded**
   Explanation: If a tx runs out of gas, execution **fails and state changes revert**, and the gas already spent **can't be reclaimed**.

7. **B.** k
   Explanation: A Merkle proof includes one sibling hash per level, so proof length is $\log_2(2^k) = k$ hashes.

8. **B. 80 bytes**
   Explanation: Bitcoin block headers are serialized in a standard **80-byte** format.

9. **B. sum of all UTXOs spendable by that address**
   Explanation: In the UTXO model, an address "balance" is the **sum of its unspent outputs** that it can spend.

10. **B. not be includable until BaseFee drops**
    Explanation: With EIP-1559, if **MaxFeePerGas < BaseFee**, the transaction can't meet the required base fee, so it's **not valid for inclusion** until the base fee falls (or it is resubmitted with a higher max fee).

Part C — Computational Problems

C1) Fee ladder with locktime

Transaction	Input (BTC)	Output to Bob (BTC)	Timelock (Height)
t_0	0.2023	**0.2022**	$T+0$
t_1	0.2023	**0.2021**	$T+2$
t_2	0.2023	**0.2020**	$T+4$
t_3	0.2023	**0.2019**	$T+6$
t_4	0.2023	**0.2018**	$T+8$
t_5	0.2023	**0.2017**	$T+10$
t_6	0.2023	**0.2016**	$T+12$
t_7	0.2023	**0.2015**	$T+14$
t_8	0.2023	**0.2014**	$T+16$
t_9	0.2023	**0.2013**	$T+18$

C2) Miner selection by fee-per-byte

1. Compute fee-per-byte

Transaction	Fee (satoshis)	Size (bytes)	Fee-per-byte (sat/byte)
C	15,000	600	**25.0**
A	8,000	400	**20.0**
D	2,800	200	**14.0**
B	3,000	250	**12.0**
E	6,000	500	**12.0**
F	900	100	**9.0**

2. Transactions Included

The block has a maximum capacity of **900 bytes**. We select transactions in descending order of fee-per-byte until the block is full.

- **Step 1: Include Tx C**
  - Size: 600 bytes
  - Total Size: 600/900 bytes
- **Step 2: Check Tx A**

- Size: 400 bytes
- Total Size if added: 600 + 400 = 1,000 bytes (**Exceeds 900 limit**)

- **Step 3: Check Tx D**
  - Size: 200 bytes
  - Total Size if added: 600 + 200 = 800 bytes (**Fits**)
- **Step 4: Check Tx B**
  - Size: 250 bytes
  - Total Size if added: 800 + 250 = 1,050 bytes (**Exceeds 900 limit**)
- **Step 5: Check Tx E**
  - Size: 500 bytes
  - Total Size if added: 800 + 500 = 1,300 bytes (**Exceeds 900 limit**)
- **Step 6: Check Tx F**
  - Size: 100 bytes
  - Total Size if added: 800 + 100 = 900 bytes (**Fits exactly**)

Transactions Included: C, D, and F.

3. Total Miner Fee Revenue

The total fee revenue is the sum of the fees from the included transactions.

- **Tx C Fee:** 15,000 satoshis
- **Tx D Fee:** 2,800 satoshis
- **Tx F Fee:** 900 satoshis

Total Miner Fee Revenue: 18,700 satoshis.

C3) Difficulty retarget

$$\text{newDifficulty} = \text{oldDifficulty} \times \frac{\text{expectedTime}}{\text{elapsedTime}} = 2000 \times \frac{20160}{16128} = 2000 \times 1.25 = 2500$$

C4) UTXO set under a fork + reorg

1. Active-chain UTXO set after B2

When **B2** arrives, the active chain is **B1** → **B2** (length 2).

- **Block B1**: Spends g_0. It creates o_1 (3.0 BTC) and o_2 (1.9 BTC).
- **Block B2**: Spends o_1. It removes o_1 from the set and creates o_3 (1.5 BTC) and o_4 (1.4 BTC).

Active UTXO Set after B2:

- o_2 (1.9 BTC to Alice)
- o_3 (1.5 BTC to Carol)
- o_4 (1.4 BTC to Bob)

2. Final active chain and UTXO set after B4

The sequence of events for the forks is as follows:

1. **B3 arrives**: It forks from B1. The current active chain (B1 → B2) has length 2. The new fork (B1 → B3) also has length 2. Since the rule requires the new chain to be **strictly longer**, the node stays on the B2 chain.
2. **B4 arrives**: It extends B3. The chain **B1** → **B3** → **B4** now has length 3. This is strictly longer than the B2 chain (length 2), so the node **switches** to this new active chain.

Final Active UTXO Set (based on B1 → B3 → B4):

- **From B1**: o_2 (1.9 BTC) remains unspent. (o_1 was created but is spent by B3).
- **From B3**: o_5 (2.9 BTC) was created but is immediately spent by B4.
- **From B4**: o_6 (2.0 BTC to Erin) and o_7 (0.8 BTC to Dave) are created.

Final UTXO Set:

- o_2 (1.9 BTC to Alice)
- o_6 (2.0 BTC to Erin)
- o_7 (0.8 BTC to Dave)

3. UTXO Status Table

This table tracks whether an output was ever part of the *active* set and its final status.

Output	Amount	Ever in Active UTXO Set?	In Final UTXO Set?	Reason
o_1	3.0	**Yes**	**No**	Created in B1; spent by B2 (initial active) and B3 (final active).
o_2	1.9	**Yes**	**Yes**	Created in B1; never spent on either chain branch.
o_3	1.5	**Yes**	**No**	Created in B2; lost when the node switched to the B3-B4 fork.
o_4	1.4	**Yes**	**No**	Created in B2; lost when the node switched to the B3-B4 fork.
o_5	2.9	**No***	**No**	Created in B3, but B3 wasn't active until B4 arrived, which spent o_5.
o_6	2.0	**Yes**	**Yes**	Created in B4; part of the final longest chain.
o_7	0.8	**Yes**	**Yes**	Created in B4; part of the final longest chain.

**Note: While o_5 exists on the B3 branch, it never technically sits in the "Active UTXO Set" as an unspent output because the moment that branch becomes active (upon B4's arrival), o_5 is already consumed by Tx4.*

Part D — Bitcoin Difficulty Adjustment

a) Difficulty Adjustment with More Miners

If more miners enter the ecosystem during a period, the difficulty parameter for the next period will **increase**.

- **Reasoning**: Increased hashing power allows the network to solve blocks faster than the target 10-minute interval.
- This results in an **elapsed time** that is **shorter** than the **expected time** (20,160 minutes).
- According to the formula provided: $\text{new_difficulty} = \text{old_difficulty} \times \frac{\text{expected_time}}{\text{elapsed_time}}$. Since the denominator (elapsed time) is smaller than the numerator (expected time), the result is a multiplier greater than 1, causing the difficulty to rise.

b) Impact of No Upper Bound on Timestamps

If there were no upper bound on accepted timestamps, an adversarial miner could **drastically decrease the mining difficulty**.

- **Mechanism**: An attacker could mine a block and set its timestamp to a date far in the future.
- Because the difficulty adjustment is calculated as $max_timestamp - min_timestamp$, a futuristic timestamp would create an artificially massive **elapsed time**.
- Plugging a very large $elapsed_time$ into the denominator of the difficulty formula would result in a **new difficulty** that is extremely low, allowing the attacker to mine future blocks with very little computational effort.

c) Difficulty Parameter at the End of $period_2$

Under the specific conditions of this attack, the difficulty parameter at the end of $period_2$ would **significantly decrease** (meaning the target becomes easier to hit).

Step-by-Step Analysis:

1. **Period 1**: The $elapsed_time$ is $t_1 - t_0$. Since t_1 is defined exactly as $t_0 + (2016 \times 10 \times 60)$ seconds, the $elapsed_time$ equals the $expected_time$. The difficulty remains **unchanged** after $period_1$.

2. **Period 2**: The adjustment is based on $t_2 - (t_0 + 2016)$.
   - t_2 is defined as $t_0 + 2 \times (2016 \times 10 \times 60)$ seconds (the time for two full periods).
   - The $elapsed_time$ calculated by the protocol for $period_2$ would be $2 \times \text{expected_time} - 2016$ seconds.

3. **Result**: Because the $elapsed_time$ is roughly **twice** the $expected_time$, the ratio $\frac{expected_time}{elapsed_time}$ will be approximately 0.5.

4. **Final Outcome**: The difficulty will **drop by approximately half**, making it much easier to mine the next period. Note: The protocol limits these adjustments to a **maximum of 4x** per cycle to prevent extreme volatility.